## Appendix 1: MATLAB Script to Optimize Masses and Spring Constants

```
% ENGN0040 Dynamics and Vibrations
% Brown University
% Project 1: Mass Launcher
% Group #28: Madison Goeke, Carter Smith, Henry Zamore, Baurice Kovatchev

clear all % Clear workspace
close all % Close all figures
clc % Clear command window

% This program calculates a set of three masses and three
% springs that will, after being dropped from a certain height, propell the
% top most mass into the air at a maximum velocity, given some constraints
% on the masses and springs, initial guesses for the masses of the
% masses and the spring constants of the springs, and estimated lengths of
% masses and springs.

% Constraints on masses and spring constants
init_vars = [.11, 5, 50, 15, 270, 830]; % Initial guesses for masses and spring constants
min_vars = [0.11,0.11,0.11,15,15,15]; % Minimum masses and spring constants
% given in instructions
max_vars = [5.84,5.84,5.84,1310,1310,1310]; % Maximum masses and spring constants
% given in instructions

% Constraint on total mass
% fmincon has the optional constraint Ax <= b. To adhere to this format,
% "A" must be a row matrix that only adds the three masses in the design
% variables, and each only once, hence the first three ones. The spring constants
% must not be included in this inequality, hence the last three zeros
A = [1,1,1,0,0,0]; % Coefficients of design variables
b = [8]; % Upper bound on matrix "A" times each design variable. This is a
% 1x1 matrix since "A", a 1x6 matrix, is to be multiplied by the the matrix
% of design variables, a 6x1 matrix, yielding a 1x1 matrix with the sum of
% component wise products

[opt_vars, opt_vel] = fmincon(@(design_vars) launch_velocity_function(design_vars), ...
    init_vars, A, b, [], [], min_vars, max_vars); % Call fmincon given the above

% listed constraints and initial guesses. Output three masses and three
% spring constants that minimize negative velocity (and therefore maximize
% positive velocity), and output that negative velocity

opt_vel = -opt_vel; % Negative to cancel out the negative placed in the ...
% "Find Launch Velocity" function needed for fmincon
opt_vel_feetpersecond = opt_vel/12; % Convert to ft/s

%% Predicted Launch Velocity Given Optimized Masses and Springs Provided
```

```
% Masses and spring constants provided closest in absolute value to those
% yielded by the optimizer
predicted_vel = -1*launch_velocity_function([.11,.829,5.84,15,134.4,959]);
% Negative to cancel out the negative placed in the "Find Launch Velocity"
% function needed for fmincon
predicted_vel_feetpersecond = predicted_vel/12; % Convert to ft/s

%% Find Launch Velocity Given 3 Masses and 3 Spring Constants
function v1 = launch_velocity_function(design_vars) % Takes in 1x6 matrix "design_vars"
% with masses and spring constants and calculates launch velocity "v1"

g = 386.0886; % Acceleration of gravity on Earth's surface (inches per second squared)
m1 = design_vars(1); % Weight of top mass is stored in the first component of design_vars
m2 = design_vars(2); % Weight of middle mass is stored in the second component of design_vars
m3 = design_vars(3); % Weight of bottom mass is stored in the third component of design_vars
k1 = design_vars(4); % Spring constant of top spring is stored in the fourth component of design_vars
k2 = design_vars(5); % Spring constant of middle spring is stored in the fifth component of design_vars
k3 = design_vars(6); % Spring constant of bottom spring is stored in the sixth component of design_vars

m1=m1/g; % Converts weight of top mass in pounds to mass in blobs by dividng by g
m2=m2/g; % Converts weight of middle mass in pounds to mass in blobs by dividng by g
m3=m3/g; % Converts weight of bottom mass in pounds to mass in blobs by dividng by g

m1l = 0.15; % Estimated length of one mass (inches)
m2l = 0.5; % Estimated length of another mass (inches)
m3l = 1; % Estimated length of a third mass (inches)
s1l = 4; % Estimated length of one spring (inches)
s2l = 4; % Estimated length of another spring (inches)
s3l = 5; % Estimated length of a third spring (inches)
total_length = m1l+m2l+m3l+s1l+s2l+s3l; % Total length of contraption (inches)
rod_length = 38; % Given in instruction (inches)
v0 = -1*sqrt(2*g*(rod_length-total_length)); % Estimated velocity just before
% contact, using the equation for velocity in freefall

% Initial conditions for ODE's
init_x1 = 0; % Initial displacement of top mass
init_x2 = 0; % Initial displacement of middle mass
init_x3 = 0; % Initial displacement of bottom mass
init_v1 = v0; % Initial velocity of top mass
init_v2 = v0; % Initial velocity of middle mass
init_v3 = v0; % Initial velocity of bottom mass
init_w = [init_x1; init_x2; init_x3; init_v1; init_v2; init_v3]; % Column vector
% with six initial conditions needed to solve six differential equations
tspan = [0, 20]; % Time thought to be much longer than the time before the event
% function ends the calculation
```

```
options = odeset('RelTol',1e-10, 'Events',@(t,w) event_function(t,w,m1,m2,m3,k1,k2,k3));
% Adjustment to ode45 function: calls event function that detects the
% separation of the top mass

[t_plot, w_plot] = ode45(@(t,w) our_ode(t, w, m1, m2, m3, k1, k2, k3, g), tspan, init_w, options);

% Call ode45 function to solve the system of differential equations given
% the initial conditions, time span, and event function

v1 = -1*w_plot(end,4); % Velocity when event function is triggered is stored in
% the last element of fourth column of w_plot. Negative because fmincon is
% a minimizer, so in order to maximize this velocity, it has to minimize
% this velocity multiplied by -1.
end

%% Separation of Top Mass Event Function
function [ev,stop,dir] = event_function(t,w,m1,m2,m3,k1,k2,k3,g) % Detects when the top
% spring is no longer contracted and stops ODE calculation
x1 = w(1); % Displacement x1 is stored in the first element of w
x2 = w(2); % Displacement x2 is stored in the second element of w.
ev = x1-x2; % Event is detected when spring 1 is decompressed. This happens
% when x1-x2 = 0
stop = 1; % Stop if event occurs
dir = 1; %  Detect only events when x1-x2 is increasing
end

%% ODE Definition
function dwdt = our_ode(t, w, m1, m2, m3, k1, k2, k3, g) % Takes in six design
% and the gravitational constant g in inches per second squared and outputs
% a matrix w as a function of time t

x1 = w(1); % Position of top mass is stored in the first component of w
x2 = w(2); % Position of middle mass is stored in the second component of w
x3 = w(3); % Position of bottom mass is stored in the third component of w
v1 = w(4); % Velocity of top mass is stored in the fourth component of w
v2 = w(5); % Velocity of middle mass is stored in the fifth component of w
v3 = w(6); % Velocity of bottom mass is stored in the sixth component of w
dv1dt = (-k1*(x1-x2) - m1*g)/m1; % Equation (1) in report
dx1dt = v1; % Equation (2) in report
dv2dt = (k1*(x1-x2) - k2*(x2-x3) - m2*g)/m2; % Equation (3) in report
dx2dt = v2; % Equation (4) in report
dv3dt = (k2*(x2-x3) - k3*x3 - m3*g)/m3; % Equation (5) in report
dx3dt = v3; % Equation (6) in report
dwdt = [dx1dt; dx2dt; dx3dt; dv1dt; dv2dt; dv3dt]; % Column vector containing
% each equation in this system of first order differential equations

end
```

## Appendix 2: MATLAB Script to Read .csv File and Plot Positions and Velocity vs. Time

```
% data_to_plots.m
data = readmatrix('mass_launcher_data.csv'); % Read .csv file

t = data(:,1); % Time is stored in first column of data
pos1 = data(:,2); % Position of top mass is stored in second column of data
pos2 = data(:,3); % Position of middle mass is stored in third column of data
pos3 = data(:,4); % Position of bottom mass is stored in fourth column of data

tv = t(1:end-1); % The velocity vector is one element shorter since it is calculated
% based on the difference of two values
vel1 = diff(pos1)/diff(t); % Calculating velocity of top mass
vel2 = diff(pos2)/diff(t); % Calculating velocity of middle mass
vel3 = diff(pos3)/diff(t); % Calculating velocity of bottom mass

figure(1) % Plot position of positions of the three masses vs. time
hold on
plot(t, pos1)
plot(t, pos2)

plot(t, pos3)
xlabel("Time (s)")
ylabel("Position (cm)")
title("Position vs. Time of Three Masses")
legend("Top Mass", "Middle Mass", "Bottom Mass")
hold off

figure(2) % Plot velocity of top mass vs. time
plot(tv, vel1)
xlabel("Time (s)")
ylabel("Velocity (cm/s)")
title("Velocity vs. Time of Top Mass")
hold off
```